completely prevents specific workspaces from exceeding predetermined thresholds, lessening the costs and increasing productivity. Also, any standards or governing policy for the cloud operating model have not been put in place yet. A policy has not been codified yet; rather, certain practices are internally known among teams and organisations. Terraform enforces sentinel policies as code before provisioning the workflow to minimise the risks through active policy enforcement.

Apply

The 'terraform apply' executes the exact same provisioning plan defined in the last step, after being reviewed. Terraform transforms configuration files *(.tf)* based on appropriate API calls to cloud provider(s) automating resource creation (using the provider's CLI) seamlessly. This will create the resources (here, an EC2 server) on AWS in a flexible and straightforward manner. Figure 5 shows the resources that will be created, and Figure 6 confirms the changes that took place.

If we proceed to the AWS console to verify the instantiation, a new EC2 instance will be up and running, as shown in Figure 7.

Destroy

After resources are created, there may be a need to terminate them. As Terraform tracks all the resources, terminating them is also simple. All that is needed is to run 'terraform destroy'. Again, Terraform will evaluate the changes and execute them after you give permission.

Additional features of Terraform

1. It provides a GUI to manage all the running services. It also provides an access control model based on the organisation, teams and users. Its audit logging emits logs whenever a change (here, change signifies sensitive write to existing IaC) happens in the infrastructure.
2. Existing pipeline integration: Terraform can be triggered from within most continuous integration/continuous deployment (CI/CD) DevOps pipelines such as Travis, Circle, Jenkins and Gitlab. This enables a plugin provisioning workflow and sentinel policies into the CI/CD pipeline.
3. Terraform supports many providers (more at *https://www.terraform.io/docs/providers/index.html*), allowing users to easily manage resources no matter where they are located. Instances can be provisioned on cloud platforms such as AWS, Azure, GCP and OpenStack using APIs provided by the cloud service providers.
4. Terraform uses a declarative style when the desired end state is written directly. Here, the tool is responsible for figuring out and achieving that end state by itself.

Let's say, we want to deploy five Elastic instances on AWS using Chef (Figure 8a) and Terraform (Figure 8b).

Observing the script given in Figure 8, one can see that both are equivalent and will produce the same results.

But let's assume a festive sale is coming up. The expected traffic will increase, and infrastructure must scale our application. Let's say, five more instances are required to handle the predicted traffic.

As language is procedural, setting count as 10 will start additional 10 instances rather than adding extra five, thus initiating a total of 15 instances. We must manually remember the previous count, as shown in Figure 9a. Hence, we must write a completely new script adding one more redundant syntax code file.

As language is declarative, setting the count as 10 (as can be seen in Figure 9b) will start an additional five

```
Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_instance.example: Creating...
aws_instance.example: Still creating... [10s elapsed]
aws_instance.example: Still creating... [20s elapsed]
aws_instance.example: Still creating... [30s elapsed]
aws_instance.example: Creation complete after 34s [id=i-0ccaeda13255269dc]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

Figure 6: Terraform showing the absolute changes made to the infrastructure

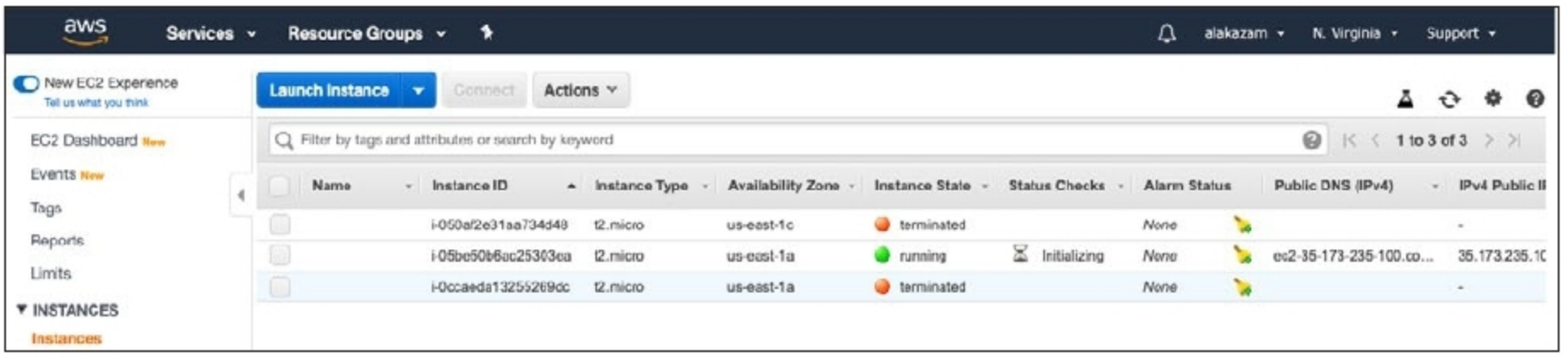

Figure 7: EC2 instance can be seen in the initialising state